Competitive Programming
Time Complexity and Prefix Sums

Sean

October 11, 2026



Outline

- Competitive programming
- Time complexity
- 1D prefix sums
- 2D prefix sums



Introduction

- Build a solving workflow and learn how judges check solutions
- Use input constraints to estimate time and memory costs
- Use prefix sums to turn repeated range scans into fast queries



1 Competitive Programming

2 Time Complexity

3 1D Prefix Sums

4 2D Prefix Sums



Competitive Programming



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits
- Each problem gives you a task, an input format, an output format, and constraints



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits
- Each problem gives you a task, an input format, an output format, and constraints
- Your goal is to design a correct algorithm, implement it, and make it fast enough



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits
- Each problem gives you a task, an input format, an output format, and constraints
- Your goal is to design a correct algorithm, implement it, and make it fast enough
- The solution is judged automatically on many hidden test cases



What a Problem Usually Contains

- Problem statement: what you need to compute
- Input format: how the data is given
- Output format: what your program must print
- Constraints: how large the input can be
- Sample tests: small examples to clarify the statement



A Good Solving Workflow

1. Understand the statement and rewrite the task in your own words



A Good Solving Workflow

1. Understand the statement and rewrite the task in your own words
2. Check the constraints and guess the needed complexity



A Good Solving Workflow

1. Understand the statement and rewrite the task in your own words
2. Check the constraints and guess the needed complexity
3. Find the algorithm or data structure



A Good Solving Workflow

1. Understand the statement and rewrite the task in your own words
2. Check the constraints and guess the needed complexity
3. Find the algorithm or data structure
4. Prove to yourself why the idea is correct



A Good Solving Workflow

1. Understand the statement and rewrite the task in your own words
2. Check the constraints and guess the needed complexity
3. Find the algorithm or data structure
4. Prove to yourself why the idea is correct
5. Implement carefully and test edge cases



Reading Constraints

- Constraints are not just technical details

Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible

Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible
- For example, checking all pairs may work for $n = 2000$

Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible
- For example, checking all pairs may work for $n = 2000$
- The same approach will usually fail for $n = 2 \times 10^5$



Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible
- For example, checking all pairs may work for $n = 2000$
- The same approach will usually fail for $n = 2 \times 10^5$
- This is why time complexity is one of the first skills to learn



Common Judge Results

- AC: accepted, the solution passed all tests
- WA: wrong answer, the program finished but produced incorrect output
- TLE: time limit exceeded, the program was too slow
- MLE: memory limit exceeded, the program used too much memory
- RE: runtime error, the program crashed during execution



Where to Practice

- CSES: excellent for structured beginner-to-intermediate practice
- Codeforces: strong contest culture, frequent rounds, and many classic problems
- AtCoder: clean statements and very high-quality problem sets
- USACO: great training path, especially for students building fundamentals step by step
- Different platforms feel different, so using more than one is helpful



How to Practice Effectively

- Solve problems regularly instead of cramming many problems in one day
- If you get stuck, think seriously first, then read the editorial and learn the key idea
- After reading a solution, try to re-implement it yourself without copying
- Revisit important problems later so the technique becomes familiar
- Keep track of patterns: prefix sums, greedy, binary search, graphs, DP, and data structures



Debugging Mindset

- When a solution fails, do not guess blindly; isolate the reason
- First decide whether the issue is correctness, time complexity, memory usage, or implementation detail
- Test small cases, edge cases, and cases where you can compute the answer by hand
- Common bugs include off-by-one errors, wrong indexing, overflow, and forgetting special cases
- A wrong answer is often easier to fix when you can produce one small counterexample



Helpful Contest Habits

- Start with a simple idea, then improve it if the constraints require it
- Test small cases by hand before trusting the code
- Pay attention to off-by-one errors and empty cases
- Use clear variable names for important quantities
- After solving, ask what pattern or technique the problem was teaching



Topics You Will See Later

- Basic data structures with STL
- Brute force and enumeration
- Searching algorithms
- Greedy algorithms
- Mathematics
- Graphs
- Dynamic programming
- Advanced data structures



1 Competitive Programming

2 Time Complexity

3 1D Prefix Sums

4 2D Prefix Sums



Time Complexity



Time Complexity

- Time complexity describes how the running time grows as the input size grows



Time Complexity

- Time complexity describes how the running time grows as the input size grows
- In competitive programming, we usually care more about growth rate than exact running time



Time Complexity

- Time complexity describes how the running time grows as the input size grows
- In competitive programming, we usually care more about growth rate than exact running time
- We use asymptotic notation such as $\mathcal{O}(1)$, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, and $\mathcal{O}(n^2)$



Time Complexity

- Time complexity describes how the running time grows as the input size grows
- In competitive programming, we usually care more about growth rate than exact running time
- We use asymptotic notation such as $\mathcal{O}(1)$, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, and $\mathcal{O}(n^2)$
- The main goal is to decide whether an algorithm is fast enough for the given constraints



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows
- A prefix array needs $\mathcal{O}(n)$ memory, a prefix grid needs $\mathcal{O}(n^2)$

Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows
- A prefix array needs $\mathcal{O}(n)$ memory, a prefix grid needs $\mathcal{O}(n^2)$
- Arrays, graphs, dynamic programming tables, and recursion depth can all affect memory usage



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows
- A prefix array needs $\mathcal{O}(n)$ memory, a prefix grid needs $\mathcal{O}(n^2)$
- Arrays, graphs, dynamic programming tables, and recursion depth can all affect memory usage
- A solution can fail even if it is fast enough when it uses too much memory



Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound



Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound
- $\Omega(f(n))$: an asymptotic lower bound

Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound
- $\Omega(f(n))$: an asymptotic lower bound
- $\Theta(f(n))$: a tight bound, meaning both upper and lower bound

Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound
- $\Omega(f(n))$: an asymptotic lower bound
- $\Theta(f(n))$: a tight bound, meaning both upper and lower bound
- In competitive programming, $\mathcal{O}(\cdot)$ is used most often in practice

What Are We Counting?

- We estimate the number of basic operations in the program



What Are We Counting?

- We estimate the number of basic operations in the program
- Fixed-size integer arithmetic and array access take $\mathcal{O}(1)$ in our model



What Are We Counting?

- We estimate the number of basic operations in the program
- Fixed-size integer arithmetic and array access take $\mathcal{O}(1)$ in our model
- Count loop iterations and the work inside each iteration



What Are We Counting?

- We estimate the number of basic operations in the program
- Fixed-size integer arithmetic and array access take $\mathcal{O}(1)$ in our model
- Count loop iterations and the work inside each iteration
- A string comparison or a container operation may do more than constant work



One Pass in Code

```cpp
vector<int> a = {3, 2, 4, 5};
long long sum = 0;
int visits = 0;
for (int x : a) {
  sum += x;                    // one addition per element
  ++visits;
}
cout << sum << ' ' << visits; // 14 4
// n visits, O(1) work each: O(n) time, O(1) extra
↪  space.
```

C++17 snippets assume `using namespace std;` and headers `<iostream>`, `<vector>`, and `<string>`.

Nested and Sequential Loops

```
1  int n = 3;
2  long long work = 0;
3  for (int i = 0; i < n; ++i) {
4    for (int j = 0; j < n; ++j) ++work; // n*n increments
5  }
6  for (int i = 0; i < n; ++i) ++work;   // n more
7  cout << work;                        // 12 = 9 + 3
8  // Total: n*n + n increments, O(n*n + n) = O(n*n).
```

Sequential costs add. Nested iteration counts multiply here because the inner loop always runs n times.



Repeated Halving in Code

```
1  int n = 16;
2  int steps = 0;
3  for (int x = n; x > 1; x /= 2) {
4    cout << x << ' ';          // 16 8 4 2
5    ++steps;
6  }
7  cout << steps;               // 4
8  // For n >= 1: floor(log2(n)) iterations, O(log n + 1).
9  // Halving the remaining work is a logarithmic pattern.
```

Contrast this with `--x`, which takes $n - 1$ iterations.



Simplifying Big-O

- In Big-O, we ignore lower-order terms

Simplifying Big-O

- In Big-O, we ignore lower-order terms
- So $\mathcal{O}(n^2 + n)$ becomes $\mathcal{O}(n^2)$

Simplifying Big-O

- In Big-O, we ignore lower-order terms
- So $\mathcal{O}(n^2 + n)$ becomes $\mathcal{O}(n^2)$
- We also ignore constant factors, so $\mathcal{O}(2n^2)$ is still $\mathcal{O}(n^2)$



Simplifying Big-O

- In Big-O, we ignore lower-order terms
- So $\mathcal{O}(n^2 + n)$ becomes $\mathcal{O}(n^2)$
- We also ignore constant factors, so $\mathcal{O}(2n^2)$ is still $\mathcal{O}(n^2)$
- This is reasonable because when n is large, the highest-order term dominates the running time



Worst Case and Amortized Analysis

- Usually we analyze the worst case



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- State which case you mean, since average-case and worst-case costs can differ



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- State which case you mean, since average-case and worst-case costs can differ
- Sometimes most operations are $\mathcal{O}(1)$, but a few rare operations are $\mathcal{O}(n)$



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- State which case you mean, since average-case and worst-case costs can differ
- Sometimes most operations are $\mathcal{O}(1)$, but a few rare operations are $\mathcal{O}(n)$
- If m operations have $\mathcal{O}(m)$ total cost, their amortized cost is $\mathcal{O}(1)$ each



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- State which case you mean, since average-case and worst-case costs can differ
- Sometimes most operations are $\mathcal{O}(1)$, but a few rare operations are $\mathcal{O}(n)$
- If m operations have $\mathcal{O}(m)$ total cost, their amortized cost is $\mathcal{O}(1)$ each
- A common example is repeated `push_back` on a `vector`



Common Complexities

- $\mathcal{O}(1)$: fixed-size arithmetic, direct array access
- $\mathcal{O}(\log n)$: binary search, repeated halving
- $\mathcal{O}(n)$: one full pass over an array
- $\mathcal{O}(n \log n)$: efficient comparison sorting
- $\mathcal{O}(n^2)$: checking all pairs
- $\mathcal{O}(2^n)$ and $\mathcal{O}(n!)$: exhaustive search on small inputs

Growth at the Input Limit

For $n = 2 \times 10^5$, compare the amount of work:

Growth	Approximate count
n	200 000
$n \log_2 n$	3.5 million
n^2	40 billion

Actual time also depends on the work per iteration, language, and judge. Estimate the growth first, then test realistic inputs.



Constraints Guide the Solution

- If n is around 2×10^5, then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable



Constraints Guide the Solution

- If n is around 2×10^5, then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable
- If n is around 2000, then $\mathcal{O}(n^2)$ may still pass



Constraints Guide the Solution

- If n is around 2×10^5, then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable
- If n is around 2000, then $\mathcal{O}(n^2)$ may still pass
- If n is around 20, then exponential search may be possible



Constraints Guide the Solution

- If n is around 2×10^5, then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable
- If n is around 2000, then $\mathcal{O}(n^2)$ may still pass
- If n is around 20, then exponential search may be possible
- A major competitive programming skill is reading the constraints and guessing the intended complexity



Optimizing an Idea

- After learning how to count complexity, the next step is to improve it



Optimizing an Idea

- After learning how to count complexity, the next step is to improve it
- Many algorithms and data structures are useful because they reduce the number of repeated operations



Optimizing an Idea

- After learning how to count complexity, the next step is to improve it
- Many algorithms and data structures are useful because they reduce the number of repeated operations
- A simple and classic example is prefix sums



1 Competitive Programming

2 Time Complexity

3 1D Prefix Sums

4 2D Prefix Sums



1D Prefix Sums



Example: Static Range Sum Queries

CSES: Static Range Sum Queries

An array has n values. Answer q queries asking for the sum of positions $[l, r]$, with both endpoints included. The array never changes. $n, q \leq 2 \times 10^5$ and $1 \leq x_i \leq 10^9$.

- Scanning one range takes up to $\mathcal{O}(n)$ time

Example: Static Range Sum Queries

CSES: Static Range Sum Queries

An array has n values. Answer q queries asking for the sum of positions $[l, r]$, with both endpoints included. The array never changes. $n, q \leq 2 \times 10^5$ and $1 \leq x_i \leq 10^9$.

- Scanning one range takes up to $\mathcal{O}(n)$ time
- Repeating the scan takes up to $\mathcal{O}(nq)$ time



Example: Static Range Sum Queries

CSES: Static Range Sum Queries

An array has n values. Answer q queries asking for the sum of positions $[l, r]$, with both endpoints included. The array never changes. $n, q \leq 2 \times 10^5$ and $1 \leq x_i \leq 10^9$.

- Scanning one range takes up to $\mathcal{O}(n)$ time
- Repeating the scan takes up to $\mathcal{O}(nq)$ time
- At the limits, that can mean 40 billion additions



A Range Scan in Code

```
// Position 0 is padding; real values are x[1] ... x[n].
vector<long long> x = {0, 3, 2, 4, 5, 1, 1, 5, 3};
auto slow_sum = [&](int l, int r) {
  long long sum = 0;
  for (int i = l; i <= r; ++i) sum += x[i];
  return sum;
};
cout << slow_sum(2, 4);     // 11 = 2 + 4 + 5
cout << slow_sum(1, 8);     // 24: scans all 8 values
// Require 1 <= l <= r <= n. Cost: O(r-l+1) per query.
```

Overlapping queries repeat the same additions.



The Prefix Table

`pre[i]` stores the sum of the first i real values.

$$pre[0] = 0, \qquad pre[i] = pre[i-1] + x[i]$$

i	0	1	2	3	4	5	6	7	8
$x[i]$	–	3	2	4	5	1	1	5	3
$pre[i]$	0	3	5	9	14	15	16	21	24

For example, $pre[4] = 3 + 2 + 4 + 5 = 14$.
The zero entry represents the empty prefix.



Building a Prefix Array in Code

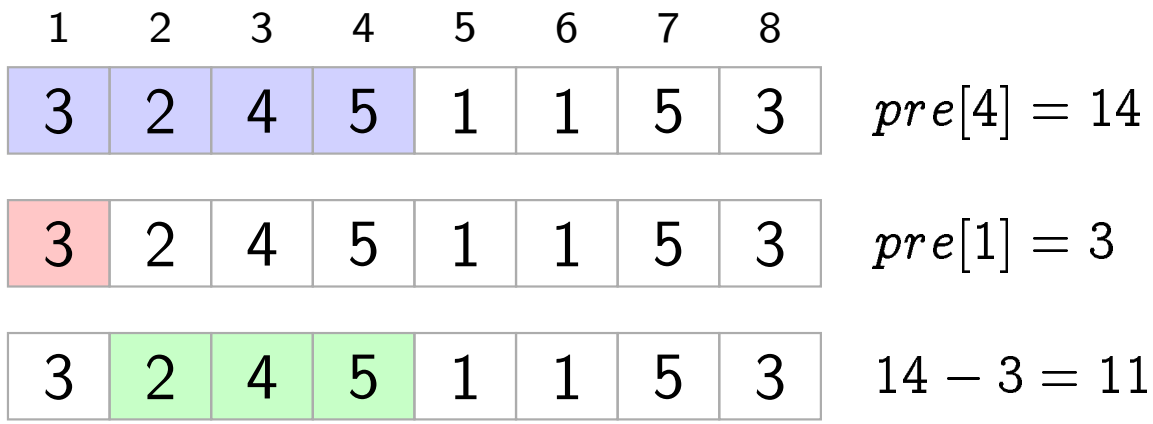
```cpp
vector<long long> x = {0, 3, 2, 4, 5, 1, 1, 5, 3};
int n = (int)x.size() - 1; // 8; x[0] is padding
vector<long long> pre(n + 1, 0); // includes pre[0]
for (int i = 1; i <= n; ++i) {
  pre[i] = pre[i - 1] + x[i];
}
for (long long s : pre) cout << s << ' ';
// 0 3 5 9 14 15 16 21 24
// pre[i] includes exactly positions 1 ... i.
// Build once: O(n) time, O(n) extra memory.
```

After step i, the table is correct through `pre[i]`.



Cancelling the Unwanted Prefix

The query $[2, 4]$ keeps positions $2, 3, 4$.



$$\text{sum}(l, r) = pre[r] - pre[l - 1]$$

The values before l appear in both prefixes and cancel.

Answering Queries in Code

```cpp
vector<long long> pre = {0,3,5,9,14,15,16,21,24};
auto range_sum = [&](int l, int r) {
  return pre[r] - pre[l - 1];
};
cout << range_sum(2, 4);   // 11: middle range
cout << range_sum(1, 8);   // 24: whole array
cout << range_sum(3, 3);   // 4: one element
cout << range_sum(1, 1);   // 3: uses pre[0]
cout << range_sum(8, 8);   // 3: last element
// Require 1 <= l <= r <= n. Each query is O(1).
// Total for n values and q queries: O(n + q).
```

Two lookups and one subtraction replace a range scan.



Indexing Conventions

Here, `pre[k]` always sums the first k values.

Range convention	**Formula**	**Same example**
1-based $[l, r]$	$pre[r] - pre[l-1]$	$[2, 4]$
0-based $[l, r]$	$pre[r+1] - pre[l]$	$[1, 3]$
0-based $[l, r)$	$pre[r] - pre[l]$	$[1, 4)$

All three examples give $pre[4] - pre[1] = 11$.
For $[l, r)$, $0 \le l \le r \le n$ and $l = r$ gives sum 0.
Choose one convention before writing the query formula.



Large Sums and Integer Types

```cpp
int n = 200000, max_value = 1000000000;
long long bound = 1LL * n * max_value;
cout << bound;                  // 200000000000000
// 1LL makes the multiplication use long long.
// long long bound = n * max_value; can overflow first.
vector<long long> x = {0, 1000000000, 1000000000,
                               1000000000};
vector<long long> pre(4, 0);
for (int i = 1; i <= 3; ++i) pre[i] = pre[i - 1] + x[i];
cout << pre[3];                 // 3000000000
// Keep the prefix table and query results in long long.
```

The CSES maximum sum is 2×10^{14}, beyond a 32-bit `int`.



Trace: Static Range Sum Queries

Values: $3, 2, 4, 5, 1, 1, 5, 3$.

Query	Right prefix	Removed prefix	Answer
$[2, 4]$	$pre[4] = 14$	$pre[1] = 3$	11
$[5, 6]$	$pre[6] = 16$	$pre[4] = 14$	2
$[1, 8]$	$pre[8] = 24$	$pre[0] = 0$	24
$[3, 3]$	$pre[3] = 9$	$pre[2] = 5$	4

These are the four queries in the CSES sample.



Contest Code: Reading and Building

```cpp
// Inside main(); continue on the next slide.
ios::sync_with_stdio(false);
cin.tie(nullptr);
int n, q;
cin >> n >> q;
vector<long long> pre(n + 1, 0);
for (int i = 1; i <= n; ++i) {
  long long x;
  cin >> x;
  pre[i] = pre[i - 1] + x;
}
// Original values are unnecessary if we only need sums.
```

Input order: `n` `q`, then the n array values.



Contest Code: The Query Loop

```cpp
// Continues with n, q, and pre from the previous slide.
for (int query = 0; query < q; ++query) {
  int l, r;
  cin >> l >> r;              // 1-based, inclusive
  cout << pre[r] - pre[l - 1] << '\n';
}
// CSES sample prints 11, 2, 24, 4 on separate lines.
// O(n + q) time, O(n) memory; no scan inside a query.
```

Input guarantees valid endpoints. The zero prefix handles $l = 1$.



Prefix Counts in Code

```cpp
vector<int> x = {0, -2, 7, 0, -3, 5}; // x[0] is padding
int n = (int)x.size() - 1;
vector<int> positive(n + 1, 0);
for (int i = 1; i <= n; ++i) {
  positive[i] = positive[i - 1] + (x[i] > 0);
}
cout << positive[5] - positive[1]; // 2: positions [2,5]
// Predicate x[i] > 0 contributes either 0 or 1.
// Negative values and zeros are valid input values.
// Prefix sums also work with negative values.
```

A range count is a sum of 0/1 indicators.



1 Competitive Programming

2 Time Complexity

3 1D Prefix Sums

4 2D Prefix Sums



2D Prefix Sums



Example: Forest Queries

CSES: Forest Queries

An $n \times n$ grid contains trees $\star$ and empty cells `.`. Count trees in each rectangle (y_1, x_1) to (y_2, x_2), including its borders. $n \leq 1000$ and $q \leq 2 \times 10^5$. The grid never changes.

- Rows y run downwards, columns x run rightwards

Example: Forest Queries

CSES: Forest Queries

An $n \times n$ grid contains trees $\star$ and empty cells `.`. Count trees in each rectangle (y_1, x_1) to (y_2, x_2), including its borders. $n \leq 1000$ and $q \leq 2 \times 10^5$. The grid never changes.

- Rows y run downwards, columns x run rightwards
- Scanning every rectangle can take $\mathcal{O}(n^2 q)$ time



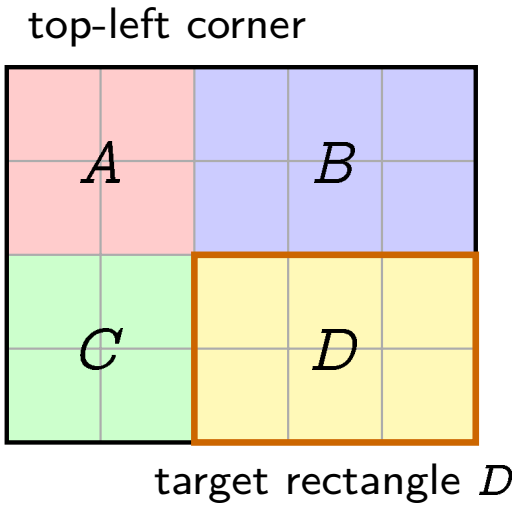
Example: Forest Queries

CSES: Forest Queries

An $n \times n$ grid contains trees $\star$ and empty cells `.`. Count trees in each rectangle (y_1, x_1) to (y_2, x_2), including its borders. $n \leq 1000$ and $q \leq 2 \times 10^5$. The grid never changes.

- Rows y run downwards, columns x run rightwards
- Scanning every rectangle can take $\mathcal{O}(n^2 q)$ time
- A prefix table gives $\mathcal{O}(n^2 + q)$ total time

Why the 2D Formula Works

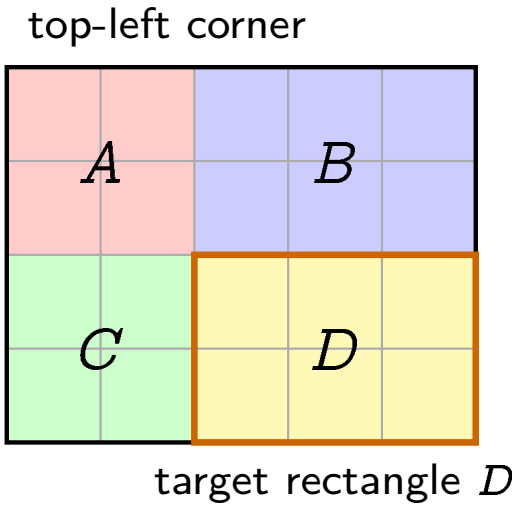


- The large prefix counts $A + B + C + D$

$$D = (A + B + C + D) - (A + B) - (A + C) + A$$

The overlap A cancels twice, so we restore one copy.

Why the 2D Formula Works

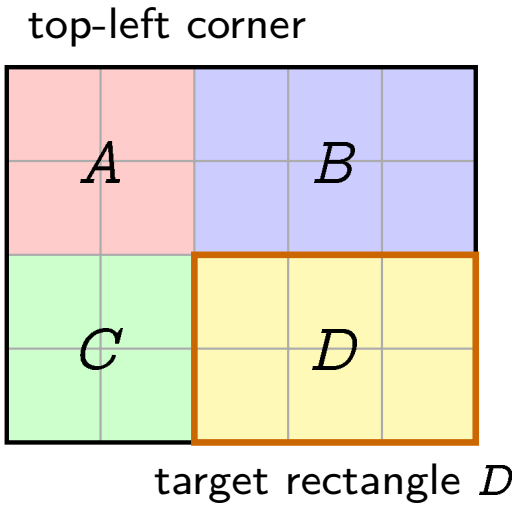


- The large prefix counts $A + B + C + D$
- Remove the area above, $A + B$

$$D = (A + B + C + D) - (A + B) - (A + C) + A$$

The overlap A cancels twice, so we restore one copy.

Why the 2D Formula Works

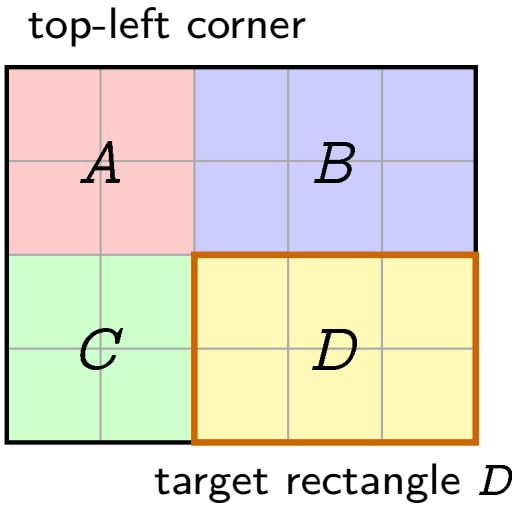


- The large prefix counts $A + B + C + D$
- Remove the area above, $A + B$
- Remove the area on the left, $A + C$

$$D = (A + B + C + D) - (A + B) - (A + C) + A$$

The overlap A cancels twice, so we restore one copy.

Why the 2D Formula Works



- The large prefix counts $A + B + C + D$
- Remove the area above, $A + B$
- Remove the area on the left, $A + C$
- Add A back once

$$D = (A + B + C + D) - (A + B) - (A + C) + A$$

The overlap A cancels twice, so we restore one copy.

Building a 2D Prefix Table

$pre[y][x]$ sums the rectangle from $(1, 1)$ through (y, x).

$$pre[y][x] = tree[y][x] + pre[y-1][x] + pre[y][x-1] - pre[y-1][x-1]$$

- `tree[y][x]` is 1 for a tree and 0 otherwise
- Allocate $(n+1) \times (n+1)$ entries, initially zero
- Row 0 and column 0 represent empty rectangles
- Fill rows from top to bottom, columns from left to right



Building the Forest Table in Code

```cpp
vector<string> forest = {".*..","*.**","**..","****"};
int n = (int)forest.size(); // 4; forest is 0-based
vector<vector<int>> pre(n + 1, vector<int>(n + 1, 0));
for (int y = 1; y <= n; ++y) {
  for (int x = 1; x <= n; ++x) {
    int tree = (forest[y - 1][x - 1] == '*');
    pre[y][x] = tree + pre[y - 1][x] + pre[y][x - 1]
                     - pre[y - 1][x - 1];
  }
}
cout << pre[4][4];          // 10: whole forest
cout << pre[2][2];          // 2: top-left 2x2
// O(n*n) build time and memory; padding stays zero.
```



Trace: The Forest Prefix Table

Tree values

0	1	0	0
1	0	1	1
1	1	0	0
1	1	1	1

`pre[y][x]` including padding

$y \backslash x$	0	1	2	3	4
0	0	0	0	0	0
1	0	0	1	1	1
2	0	1	2	3	4
3	0	2	4	5	6
4	0	3	6	8	10

$$pre[3][2] = 1 + pre[2][2] + pre[3][1] - pre[2][1]$$

$1 + 2 + 2 - 1 = 4$ trees in the first 3 rows and 2 columns.



Answering a Rectangle Query

For inclusive corners (y_1, x_1) and (y_2, x_2):

$$\begin{aligned} ans = & \, pre[y_2][x_2] \\ & - pre[y_1 - 1][x_2] \\ & - pre[y_2][x_1 - 1] \\ & + pre[y_1 - 1][x_1 - 1] \end{aligned}$$

- Four lookups give $\mathcal{O}(1)$ time per query
- The zero padding handles top and left borders
- Require $1 \leq y_1 \leq y_2 \leq n$ and $1 \leq x_1 \leq x_2 \leq n$



Rectangle Queries in Code

```cpp
// Uses pre from the forest build slide.
auto rectangle = [&](int y1, int x1, int y2, int x2) {
  return pre[y2][x2] - pre[y1 - 1][x2]
       - pre[y2][x1 - 1] + pre[y1 - 1][x1 - 1];
};
cout << rectangle(2, 2, 3, 4); // 3: CSES sample
cout << rectangle(3, 1, 3, 1); // 1: one tree cell
cout << rectangle(1, 1, 2, 2); // 2: top and left
 ↪  borders
cout << rectangle(1, 1, 4, 4); // 10: entire forest
cout << rectangle(1, 1, 1, 1); // 0: one empty cell
cout << rectangle(3, 3, 4, 4); // 2: nonzero overlap
// 1-based inclusive corners: row, column, row, column.
```



Total: $\mathcal{O}(n^2 + q)$ time and $\mathcal{O}(n^2)$ memory.

Trace: A Forest Query

The query $(2, 2)$ to $(3, 4)$ includes two rows and three columns.

Term	Region	Value
$pre[3][4]$	Large prefix	6
$-pre[1][4]$	Remove rows above	-1
$-pre[3][1]$	Remove columns left	-2
$+pre[1][1]$	Restore overlap	0

$$6 - 1 - 2 + 0 = 3$$

For $(3, 3)$ to $(4, 4)$, the answer is $10 - 4 - 6 + 2 = 2$.



Prefix Sum Pitfalls

- Allocate the zero prefix, and the zero row and column in 2D
- Match the query formula to the indexing convention
- Size the numeric type for the largest possible total
- In 2D, subtract both strips and add the overlap back
- Rebuild or change the data structure when values change

A change to position k changes every prefix from k onwards.



Choosing a Range Query Technique

Task	**Useful approach**
One sum or a short range	Direct scan
Many sums on a fixed array	1D prefix sums
Many counts on a fixed grid	2D prefix sums
Counts of a condition	Prefix sums of 0/1 values
Updates and range sums	Fenwick or segment tree
Range minima	A different range data structure

Sums support cancellation by subtraction. Prefix minima lose information about the values inside a range.



Takeaways

- Constraints guide the time and memory budget
- Count iterations and include the cost of the loop body
- Prefix sums reuse work across overlapping queries
- Zero padding makes boundary queries use the same formula
- Check single elements, full ranges, borders, and large totals



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

